

Does `/init` Help?

A Meta-Study of Repository Context Files for Coding Agents

Marcel Petrick · Claude Opus 5

19 September 2026 · revised 20 September 2026

Abstract. Agentic coding harnesses ship commands that generate a repository context file—`CLAUDE.md`, `AGENTS.md`—which is then loaded into every subsequent session. We synthesise the five controlled ablations of this practice, the surrounding mechanism literature, and primary sources extracted directly from a shipped Claude Code binary. Every measured effect on task success lies *within* the $\approx 9\%$ of per-instance outcomes that flip between byte-identical runs at temperature zero—so the single-run designs, which supply most of the field’s point estimates, do not distinguish the practice from no practice. The designs that repeat their trials mostly report nulls, with one exception that locates the effect precisely: guidance *tuned against an agent’s observed failures* lifts resolve rate $25.5 \rightarrow 33.0\%$ on an open 35B model ($p < 0.001$), and does so by helping the agent reach the right file, not by improving its patches. The one robust effect across the frontier agents is cost, at $+20\text{--}23\%$. Decomposing the artefact resolves the apparent disagreement: *instructions* are followed (a named tool is invoked $1.6\times$ per task versus $<0.01\times$ unmentioned) whereas *repository overviews* are not helpful, because natural-language summaries of code answer 4/45 behavioural questions where the source answers 27/45—a property of the representation, not of the author. We argue the pattern follows a general principle: **scaffolding decays when it substitutes for a capability the model now has natively, and persists when it supplies information the model cannot derive or enforces a constraint it cannot infer.**

1 Introduction

The practice is near-universal and the evidence is thin. Every major harness (Claude Code, Codex, Copilot, Cursor, Aider, Gemini CLI, Windsurf, Cline, Amp, Devin, Junie) ships an instruction-file mechanism; essentially none of their vendors has published a controlled measurement of whether it works. The entire causal base is five 2026 preprints, none replicated, none formally rebutted.

We ask three questions. Does a context file improve task success? What does it cost? And if the answers are “not measurably” and “20%,” why does the feature exist?

2 Method

Five rounds of literature search (three of them ten parallel agents) were combined with two local primary-source efforts: extraction of the literal `/init` prompt, an unreleased gated successor, and the loading semantics from the shipped Claude Code binary v2.1.278; and measurement of 35 instruction files in a working repository corpus (median $\approx 1,590$ tokens, $p90 \approx 3,300$).

Agent output was not trusted. Every load-bearing claim was re-fetched from the primary source. This caught two passes reporting mutually incompatible figures from one paper body, one pass reporting p -values absent from the paper it cited, two fabricated statistics in circulation, and one vendor figure absent from its own source. A full verification log and a self-review of our own first draft accompany this paper.

3 Results

3.1 Most point estimates are not measurements

Figure 1 plots every measured effect on task success against the measured run-to-run noise floor. Sam-Bodden [2], under a protocol frozen before data collection, reports that temperature-0 inference “flips $\approx 9\%$ of per-instance outcomes between byte-identical runs,” and states this is “a noise floor under every small effect reported on this benchmark.” Every published point estimate falls inside it, and the study contributing several of them sampled each instance once, with no significance testing.

The band must be read for what it is. $\approx 9\%$ is a *per-instance*

Table 1: The complete causal evidence base. All five are 2026 preprints or workshop papers; none is replicated or formally rebutted. “Repeats” denotes multiple runs of the same condition—the column that decides whether a point estimate is a measurement.

Study	Scale	Repeats	Outcome
Gloaguen [1]	438 instances, 4 agents	No (1 $\times$)	No success gain; +20–23% cost
Khatr [3]	288 runs, 17 tasks	Yes (3 $\times$)	No effect; but MDE ≈ 30 pp
Lulla [4]	124 PRs, 10 repos	No (paired)	–28.6% runtime; correctness spot-checked
McMillan [5]	1,650 sessions	Yes (50 $\times$)	Affirmative null on file structure
Shepard [6]	4 trials, Qwen3.5-35B	Yes (4 $\times$)	+7.5 pp, $p < 0.001$; <i>tuned</i> guidance only

flip rate—what one run establishes about one instance—not a minimum detectable effect. Repeated trials shrink the standard error of the mean and can resolve a smaller difference, which is why the one positive result below survives inside the band. The floor disqualifies the single-run estimate, not the practice.

This is corroborated independently: single runs correlate only $\rho = 0.417$ with true repeated-rollout reliability [12], and run-level pass rates overstate retry-free coverage by up to 17.8 pp [13]. A five-run replication of one comparison found a single run showing 44% *slower* where the five-run median showed 27% *faster*.

3.2 Cost is the one robust effect

Gloaguen et al. [1] find inference cost rises 20% (SWE-bench Lite) to 23% (AGENTbench), with 2–4 additional reasoning steps, across every model tested (Figure 2). Lulla et al. [4] report the opposite sign—median runtime –28.6%, output tokens –16.6%—but on PRs restricted to ≤ 100 LOC, with correctness spot-checked on only 50 of 124 cases, leaving “did less work” unexcluded.

Cost in tokens is not cost in currency. `CLAUDE.md` occupies

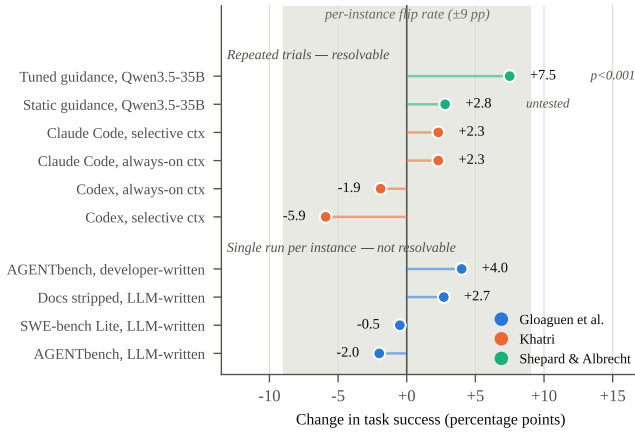


Figure 1: Every measured change in task success, grouped by whether the design can resolve one. The band is the ± 9 pp per-instance flip rate [2]; no effect escapes it, but that only settles the four single-run estimates. Of the six from designs with repeats, four are bounded nulls, one (+2.8 pp) is never tested, and one (+7.5 pp, $p < 0.001$) is real.

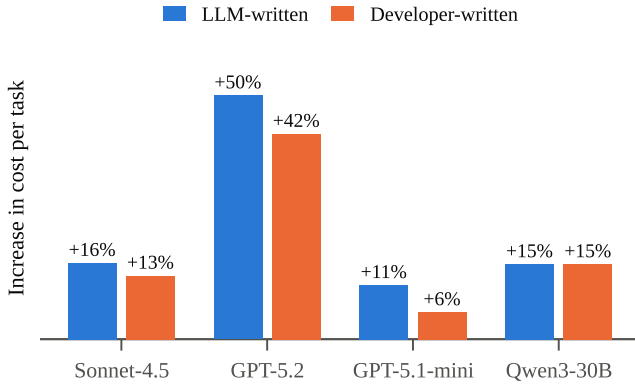


Figure 2: Increase in cost per task on AGENTbench when a context file is present, by model and provenance. Derived from Table 2 of [1]. Unlike the success-rate effects, this is consistent in sign across all four models.

Claude Code’s cached project-context layer; at published rates a 10,000-token file costs \$0.12 across a 50-turn session, and the cached/uncached ratio is exactly $50/6.15 = 8.13\times$, independent of file size. *The monetary argument against a large context file does not survive contact with the arithmetic.*

3.3 Two artefacts, opposite signs

The decomposition in [1] resolves the field’s apparent contradiction: “instructions in the context files are well followed by coding agents, repository overviews ... are not helpful.” Instructions move behaviour—a tool named in the file is invoked $1.6\times$ per instance versus $< 0.01\times$ unmentioned; repository-specific tools $2.5\times$ versus $< 0.05\times$. Overviews do not measurably reduce steps-to-first-relevant-file, despite appearing in 100% of LLM-generated files.

Sam-Bodden [2] supplies the mechanism and forecloses the obvious remedy: prose summaries answer 4/45 behavioural questions about a file where the source answers 27/45, and “the gap belongs to the representation, not the summarizer—a frontier model’s summaries score exactly as poorly as a 3B model’s.” A registered hypothesis that structured skeletons would help *failed* ($N=70$, McNemar $p=0.75$). Figure 3 shows that the category measured not to help is present in roughly two-thirds of real files.

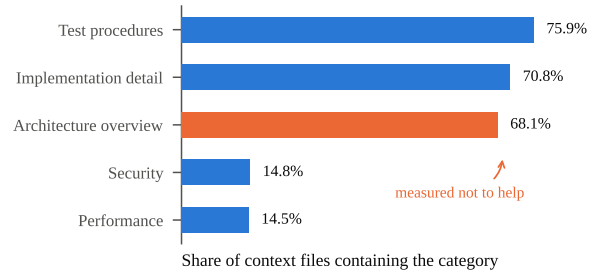


Figure 3: Content categories across 2,303 context files [11]. Architecture overviews—the one category with direct evidence of no benefit [1, 2]—appear in 68.1% of files.

3.4 The exception: tuned guidance on a weaker model

One design breaks the pattern, and it is the most careful of the five. Shepard and Albrecht [6] introduce *probe-and-refine tuning*: synthetic bug-fix probes iteratively diagnose and patch a repository’s guidance file through single-shot calls, with no agent loop during tuning. Over four independent trials on SWE-bench Verified with Qwen3.5-35B-A3B, resolve rate runs 25.5% unguided, 28.3% with a static knowledge base, and **33.0%** tuned ($p < 0.001$ for both tuned contrasts). They conclude that “how the guidance is produced is the decisive variable.”

Three things keep this compatible with the nulls. The model is an open 35B, not a frontier agent—and their cross-model check finds the procedure *degrades* on a model too weak to emit diagnostic output, so the scaffolding has a capability window on both sides. The static knowledge base, the condition closest to what `/init` writes, gained 2.8 pp, which the paper does not significance-test; the demonstrated gain required tuning against observed failures, making the file a defect log rather than a repository tour. And the improvement is located: coverage rises 14.5 pp while per-patch precision stays flat ($\approx 59\%$, $p = 0.119$), so guidance “helps agents reach the correct file rather than improving the quality of the changes they make”—the nulls’ conclusion, reached from the opposite side.

3.5 The artefact has no brake

Across 247,694 instruction lifetimes in 1,867 repositories, context files grow **+226%** over their lifetime, a net +4.9 instructions per commit, and the older an instruction becomes the *less* likely it is ever deleted [8]. Appending is cheap; verifying a deletion is safe is $O(2^{|D|})$. Independently, 23.0% of such files already contain stale code references [9], and a second mining study of 1,540 rule-evolution events finds evolution dominated by expansions (29.17%) and enrichments (26.59%), with developers reporting that they fix agent errors “by adding new negative constraints rather than editing existing ones” [7]. That study also supplies the counterweight: after a rule update, artefact compliance rises from 49.14% to 72.13%. Structural file properties—size, position, architecture, internal contradiction—show *affirmative* null effects on adherence ($BF_{10} = 0.096, 0.053$); what degrades compliance is session length, at -5.6% odds per generated function [5].

4 Discussion: why the command exists

`/init` shipped in Claude Code’s first public build (2025-02-24), so it was not a patch for an observed deficiency. But its performance rationale has eroded in step with the harness acquiring cheaper homes for the same content: hooks

(2025-06), subagents (2025-07), Skills and a context-saving Explore subagent (2025-10), path-scoped rules (2025-12), auto-memory (2026-02). Vendor framing tracks this exactly, from “an ideal place” for documentation with no size limit (2025-04), to the “naively dropped into context up front” half of a hybrid with just-in-time retrieval (2025-09), to a hard 200-line target with the warning that “bloated CLAUDE.md files cause Claude to ignore your actual instructions” (2026-09). An unreleased successor to `/init`, recovered from the binary, opens: “CLAUDE.md is loaded into every Claude Code session, so it must be concise—only include what Claude would get wrong without it.”

The feature also carries rationales that cannot decay: it migrates Cursor, Copilot, Windsurf, Devin and Cline rule files; it onboards new users to the memory system; and it produces a human-readable artefact for the team.

A principle

Derived from an unrelated literature—chain-of-thought [14], prefill, verification prompting, minimal agent scaffolds [15]—and found to predict the present result:

Scaffolding decays when it substitutes for a capability the model now has natively; it persists when it supplies information the model structurally cannot derive, or enforces a constraint it cannot infer.

A repository overview substitutes for reading the codebase—and the agent can now grep. Commands, gotchas and prohibitions supply what no amount of reading yields. The halves have opposite fates because they are different kinds of thing. The principle’s sharpest test is §3.5: the same artefact is worth +7.5 pp to a 35B model that cannot reliably find the right file and nothing measurable to agents that can.

5 Verdict

The practice is sound; the default artefact is not. Run `/init` as a draft, then delete the architecture tour—it is the half measured not to help, and it cannot be fixed by rewriting it. Keep only what the repository cannot state about itself. Count imperatives rather than lines, since compliance degrades with constraint count and session length, not file size. Move enforcement to hooks, procedures to skills, and module detail to path-scoped files; `@imports` load eagerly and save nothing. Do not shrink the file to save money—shrink it to protect compliance. Record what the agent got *wrong* rather than what the repository contains—that is the one intervention with a significance-tested gain behind it, and the test Anthropic’s own unreleased `/init` applies. Expect no gain in success rate on a frontier agent: they fail on implementation skill, which no document repairs. Weight this differently on a smaller or local model—the only measured gain in the literature is on an open 35B.

Limitations. Five unreplicated preprints, three underpowered for the effects they report; no experiment of our own; modelled rather than measured cost tables; a single binary version; and a search budget exhausted before the final round. A later sweep found two relevant papers the first rounds had missed, one of which changed a headline claim—assume the same of this version. The decisive experiment—ablating instructions against overviews under one shared success metric—remains unrun, and is cheap.

6 Open experiments

The literature’s gaps are specific, and several are inexpensive to close.

1. **Halve the file.** Ablate *instructions only* against *overview only* against *both* and *neither*, under one shared success metric with repeated runs. Every ingredient exists; the released AGENTbench harness makes it a week of compute. This is the decisive experiment and it has not been run.
2. **Measure all three at once.** No study reports success, cost and latency together with repeats. Given a ≈ 9 pp noise floor [2], a design powered for a 5 pp effect needs roughly 120–200 tasks [3]—large, but not exotic.
3. **Staleness versus consequence.** Prevalence is known (23.0% of files carry stale references [9]); the performance cost of that staleness is not.
4. **The capability window.** Guidance pays on a 35B model [6] and not on frontier agents, but no design spans a capability ladder. “Does it help?” is missing its qualifier: help whom? Relatedly, probe-and-refine has never been run on a frontier agent—the cheapest open experiment of real value.
5. **Generalisation.** The evidence is overwhelmingly Python and TypeScript. Gloaguen et al. themselves caution that Python’s weight in training data “might... nullify the effect of context files” [1]—the practice may well pay in languages the model knows less well.

References

- [1] T. Gloaguen, N. Mündler, M. Müller, V. Raychev, M. Vechev. *Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?* MemAgents @ ICLR 2026. arXiv:2602.11988.
- [2] B. Sam-Bodden. *What Context Does a Coding Agent Actually Need to Act?* arXiv:2607.09691, 2026.
- [3] P. Khatri. *Do Context Files Help Coding Agents? A Two-Agent Ablation Study on Real Repositories.* arXiv:2607.27250, 2026.
- [4] J. L. Lulla et al. *On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents.* JAWs @ ICSE 2026. arXiv:2601.20404.
- [5] D. McMillan. *Instruction Adherence in Coding Agent Configuration Files.* arXiv:2605.10039, 2026.
- [6] A. Shepard, J. Albrecht. *Probe-and-Refine Tuning of Repository Guidance for Coding Agents.* arXiv:2606.20512, 2026.
- [7] G. Cai, R. Li, P. Liang, Z. Li, M. Shahin. *Rule Taxonomy and Evolution in AI IDEs: A Mining and Survey Study.* arXiv:2606.12231, 2026.
- [8] K. Chakrabarti. *Why Does CLAUDE.md Keep Growing? Catastrophic Remembering in Agentic Coding.* arXiv:2608.11095, 2026.
- [9] C. Treude, S. Baltes. *Context Rot in AI-Assisted Software Development.* arXiv:2606.09090, 2026.
- [10] A. Arabat, M. Sayagh. *Toward Instructions-as-Code.* MSR 2026. arXiv:2606.13449.
- [11] P. Chatlatanagulchai et al. *Agent READMEs: An Empirical Study of Context Files for Agentic Coding.* arXiv:2511.12884.
- [12] Y. Jiang et al. *Beyond Pass@k.* arXiv:2608.14711, 2026.
- [13] *Accuracy, Stability, and Repeated-Run Reliability of LLMs on Deterministic Programming Tasks.* arXiv:2606.00920, 2026.
- [14] Z. Sprague et al. *To CoT or not to CoT?* ICLR 2025. arXiv:2409.12183.
- [15] C. Xia et al. *Agentless: Demystifying LLM-based Software Engineering Agents.* arXiv:2407.01489.